

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC BÁCH KHOA**  
**KHOA ĐIỆN – ĐIỆN TỬ**



**BÀI TẬP HỌC PHẦN MỞ RỘNG**  
**KỸ THUẬT SỐ (EE100D)**

**GVHD: ThS. Phan Võ Kim Anh**

**Nhóm 03 – Lớp TND2**

| STT | MSSV    | Họ và tên đệm    | Tên   | Ghi chú |
|-----|---------|------------------|-------|---------|
| 1   | 2410019 | Nguyễn Đặng Thái | An    |         |
| 2   | 2410626 | Lương Thanh      | Dương |         |
| 3   | 2410681 | Lê Tuấn          | Đạt   |         |
| 4   | 2411835 | Lý Tùng          | Lâm   |         |

*Thành phố Hồ Chí Minh – tháng 1 năm 2026*



# MỤC LỤC

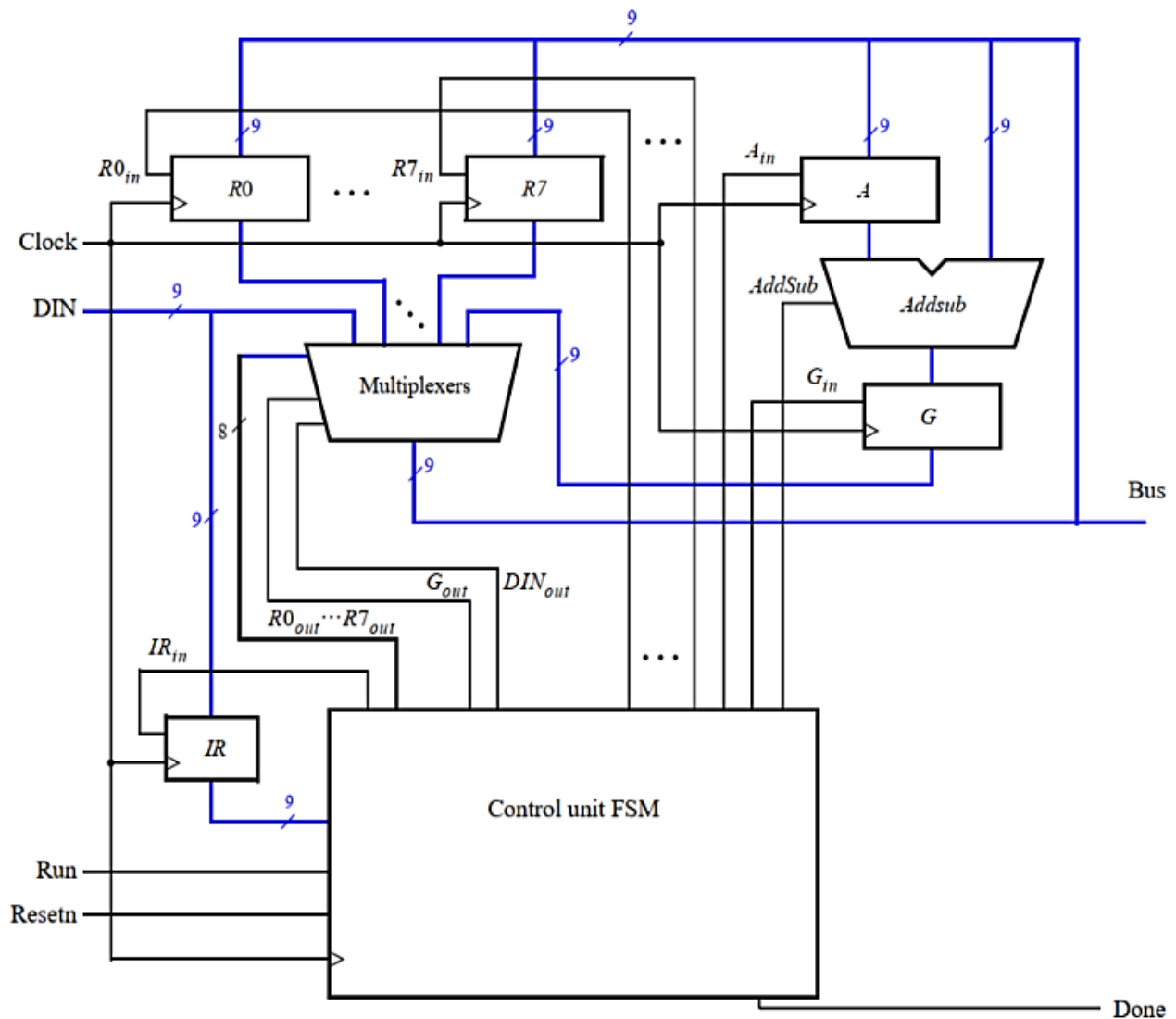
|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>EXPERIMENT 1: Design and implement a simple processor</b>             | <b>1</b>  |
| 1. Yêu cầu bài toán                                                      | 1         |
| 2. Thiết kế                                                              | 2         |
| 3. Kết quả mô phỏng sóng (Waveform)                                      | 7         |
| 4. RTL Viewer                                                            | 8         |
| <b>EXPERIMENT 2: Design and implement a simple processor with memory</b> | <b>9</b>  |
| 1. Yêu cầu bài toán                                                      | 9         |
| 2. Thiết kế                                                              | 9         |
| 3. Kết quả mô phỏng sóng (Waveform)                                      | 12        |
| 4. RTL Viewer                                                            | 13        |
| <b>KẾT LUẬN</b>                                                          | <b>14</b> |

## LABORATORY 4: A SIMPLE PROCESSOR

### EXPERIMENT 1: Design and implement a simple processor

#### 1. Yêu cầu bài toán

Thiết kế và hiện thực một bộ xử lý đơn giản như sơ đồ trong Hình 1 bằng SystemVerilog. Hệ thống gồm các khối chức năng gồm các thanh ghi 9 bit, bộ dồn kênh Multiplexers, khối tính toán ALU (AddSub) và khối điều khiển trung tâm FSM. Bộ xử lý phải có khả năng giải mã và thực thi chính xác 4 lệnh cơ bản: mv, mvi, add và sub. Sau đó, kiểm chứng thiết kế thông qua mô phỏng dạng sóng và sơ đồ RTL.



Hình 1: Sơ đồ bộ xử lý đơn giản

## 2. Thiết kế

### a. Module dec3to8 (Decoder 3 – 8)

Dùng để giải mã các trường địa chỉ thanh ghi trong mã máy (ví dụ: 3 bit XXX hoặc YYY trong lệnh). Đảm bảo chỉ có đúng một thanh ghi được chọn để đọc hoặc ghi tại một thời điểm dựa trên mã lệnh.

```
1 module dec3to8(  
2     input logic [2:0] W,  
3     input logic En,  
4     output logic [7:0] Y  
5 );  
6     always_comb begin  
7         if (En == 1'b1) begin  
8             case (W)  
9                 3'b000: Y = 8'b00000001;  
10                3'b001: Y = 8'b00000010;  
11                3'b010: Y = 8'b00000100;  
12                3'b011: Y = 8'b00001000;  
13                3'b100: Y = 8'b00010000;  
14                3'b101: Y = 8'b00100000;  
15                3'b110: Y = 8'b01000000;  
16                3'b111: Y = 8'b10000000;  
17                default: Y = 8'b00000000;  
18            endcase  
19        end else begin  
20            Y = 8'b00000000;  
21        end  
22    end  
23 endmodule
```

### b. Module mux (Multiplexers)

Điều hướng dữ liệu trong hệ thống. Nó chọn một trong nhiều nguồn dữ liệu đầu vào để đưa lên đường truyền chung (Bus).

```
1 module mux(  
2     input logic [8:0] R_in [0:7],  
3     input logic [8:0] G_in, DIN_in,  
4     input logic [7:0] R_out,  
5     input logic G_out, DIN_out,  
6     output logic [8:0] Bus_out  
7 );  
8     always_comb begin  
9         if(DIN_out) Bus_out = DIN_in;  
10        else if(R_out[0]) Bus_out = R_in[0];  
11        else if(R_out[1]) Bus_out = R_in[1];  
12        else if(R_out[2]) Bus_out = R_in[2];  
13        else if(R_out[3]) Bus_out = R_in[3];  
14        else if(R_out[4]) Bus_out = R_in[4];  
15        else if(R_out[5]) Bus_out = R_in[5];
```

```

16                                     else if(R_out[6]) Bus_out = R_in[6];
17                                     else if(R_out[7]) Bus_out = R_in[7];
18                                     else if(G_out) Bus_out = G_in;
19                                     else Bus_out = 9'b000000000;
20     end
21 endmodule

```

### c. Module regn (Register)

Các thanh ghi là nơi lưu trữ dữ liệu tạm thời.

- R0 - R7: Các thanh ghi dùng để lưu trữ các biến hoặc dữ liệu trong quá trình xử lý.
- IR: Thanh ghi lưu trữ lệnh hiện tại đang được thực thi (lấy từ DIN).
- A: Thanh ghi lưu trữ số hạng thứ nhất trước khi thực hiện phép tính cộng hoặc trừ.
- G: Thanh ghi lưu trữ kết quả đầu ra từ bộ cộng/trừ (AddSub).

```

1 module regn #(parameter n=9) (
2     input logic [n-1:0] R,
3     input logic Rin, Clock,
4     output logic [n-1:0] Q
5 );
6
7     always_ff @(posedge Clock)
8         if (Rin) Q <= R;
9 endmodule

```

### d. Module AddSubUnit (ALU)

Dùng thực hiện phép cộng hoặc phép trừ giữa hai số 9-bit. Tín hiệu AddSub sẽ quyết định thực hiện phép cộng hay phép trừ. Kết quả sau tính toán được đưa vào thanh ghi G.

```

1 module AddSubUnit (
2     input logic [8:0] A_in, Bus_in,
3     input logic AddSub,
4     output logic [8:0] AddSub_out
5 );
6     always_comb begin
7         if (AddSub == 1'b0)
8             AddSub_out = A_in + Bus_in;
9         else
10            AddSub_out = A_in - Bus_in;
11        end
12    endmodule

```

### e. Module CPU (Top – level)

Đây là toàn bộ hệ thống xử lý (Processor). Nó đóng vai trò là "bộ não" kết nối và điều phối hoạt động giữa Khối điều khiển (Control Unit) và Khối đường dữ liệu (Datapath). Nhận tín

hiệu đầu vào (Clock, Resetn, Run, DIN), thực hiện việc tìm nạp lệnh, giải mã lệnh và thực thi lệnh (di chuyển dữ liệu, tính toán cộng/trừ) thông qua hệ thống Bus 9-bit, sau đó báo hiệu khi hoàn tất qua tín hiệu Done.

```

1 module CPU (
2   input logic [8:0] DIN,
3   input logic Resetn, Clock, Run,
4   output logic Done,
5   output logic [8:0] BusWires,
6       output logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7, IR, A, G,
7       output logic [1:0] prestate
8 );
9     logic [8:0] R [0:7];
10    assign R0 = R[0];
11    assign R1 = R[1];
12    assign R2 = R[2];
13    assign R3 = R[3];
14    assign R4 = R[4];
15    assign R5 = R[5];
16    assign R6 = R[6];
17    assign R7 = R[7];
18    logic Ain, Gin, IRin;
19    logic [7:0] R_in;
20    logic [7:0] R_out;
21    logic G_out, DIN_out;
22    logic AddSub_ctrl;
23    logic [8:0] AddSub_out;
24    typedef enum logic [1:0] {T0, T1, T2, T3} states;
25    states Tstep_Q, Tstep_D;
26    logic [2:0] I;
27    logic [7:0] Xreg, Yreg;
28    // Các Thanh ghi (Registers)
29    regn u_IR (.R(DIN), .Rin(IRin), .Clock(Clock), .Q(IR));
30    regn u_A (.R(BusWires), .Rin(Ain), .Clock(Clock), .Q(A));
31    regn u_G (.R(AddSub_out), .Rin(Gin), .Clock(Clock), .Q(G));
32    regn u0 (.R(BusWires), .Rin(R_in[0]), .Clock(Clock), .Q(R[0]));
33    regn u1 (.R(BusWires), .Rin(R_in[1]), .Clock(Clock), .Q(R[1]));
34    regn u2 (.R(BusWires), .Rin(R_in[2]), .Clock(Clock), .Q(R[2]));
35    regn u3 (.R(BusWires), .Rin(R_in[3]), .Clock(Clock), .Q(R[3]));
36    regn u4 (.R(BusWires), .Rin(R_in[4]), .Clock(Clock), .Q(R[4]));
37    regn u5 (.R(BusWires), .Rin(R_in[5]), .Clock(Clock), .Q(R[5]));
38    regn u6 (.R(BusWires), .Rin(R_in[6]), .Clock(Clock), .Q(R[6]));
39    regn u7 (.R(BusWires), .Rin(R_in[7]), .Clock(Clock), .Q(R[7]));
40    assign I = IR[8:6];
41    dec3to8 u_decX (
42      .W(IR[5:3]),
43      .En(1'b1),
44      .Y(Xreg)
45    );
46    // Decoders (YYY)
47    dec3to8 u_decY (
48      .W(IR[2:0]),
49      .En(1'b1),

```

```

50     .Y(Yreg)
51 );
52
53 // AddSub Unit
54 AddSubUnit u_addsub (
55     .A_in(A),
56     .Bus_in(BusWires),
57     .AddSub(AddSub_ctrl),
58     .AddSub_out(AddSub_out)
59 );
60
61 // FSM State Register (Control FSM flip-flops)
62 always_ff @(posedge Clock, negedge Resetn) begin
63     if (!Resetn)
64         Tstep_Q <= T0;
65     else
66         Tstep_Q <= Tstep_D;
67 end
68     mux u_bus_mux (
69         // Đầu vào dữ liệu
70         .R_in(R),    // [8:0] R[0:7] (Q outputs of R0-R7)
71         .G_in(G),    // [8:0] G (Q output of G register)
72         .DIN_in(DIN), // [8:0] DIN (Top-level input)
73
74         // Đầu vào điều khiển (từ FSM)
75         .R_out(R_out), // [7:0] R_out (Control signals for R0-R7)
76         .G_out(G_out), // [1:0] G_out (Control signal for G)
77         .DIN_out(DIN_out), // [1:0] DIN_out (Control signal for DIN)
78
79         // Đầu ra Bus
80         .Bus_out(BusWires) // [8:0] BusWires (Top-level output)
81 );
82 // FSM Next State Logic (Control FSM state table)
83 always_comb begin
84     Tstep_D = Tstep_Q;
85
86     case (Tstep_Q)
87         T0:
88             if (!Run) Tstep_D = T0;
89             else Tstep_D = T1;
90
91         T1:
92             case (I)
93                 3'b000: Tstep_D = T0;
94                 3'b001: Tstep_D = T0;
95                 3'b010: Tstep_D = T2;
96                 3'b011: Tstep_D = T2;
97                 default: Tstep_D = T0;
98             endcase
99
100        T2:
101            Tstep_D = T3;
102
103        T3:
104            Tstep_D = T0;

```

```

105
106     default: Tstep_D = T0;
107 endcase
108 end
109
110 // FSM Output Logic (Control FSM outputs)
111 always_comb begin
112     // Specify initial values
113     IRin = 1'b0;
114     Ain = 1'b0;
115     Gin = 1'b0;
116     R_in = 8'b00000000;
117     R_out = 8'b00000000;
118     G_out = 1'b0;
119     DIN_out = 1'b0;
120     AddSub_ctrl = 1'b0;
121     Done = 1'b0;
122
123     case (Tstep_Q)
124         T0: begin
125             IRin = Run;
126             DIN_out = 1'b1;
127         end
128
129         T1:
130             case (I)
131                 3'b000: begin
132                     R_out = Yreg;
133                     R_in = Xreg;
134                     Done = 1'b1;
135                 end
136                 3'b001: begin
137                     DIN_out = 1'b1;
138                     R_in = Xreg;
139                     Done = 1'b1;
140                 end
141                 3'b010: begin
142                     R_out = Xreg;
143                     Ain = 1'b1;
144                 end
145                 3'b011: begin
146                     R_out = Xreg;
147                     Ain = 1'b1;
148                 end
149                 default: ;
150             endcase
151
152         T2:
153             case (I)
154                 3'b010: begin
155                     R_out = Yreg;
156                     Gin = 1'b1;
157                     AddSub_ctrl = 1'b0; // Add
158                 end
159                 3'b011: begin

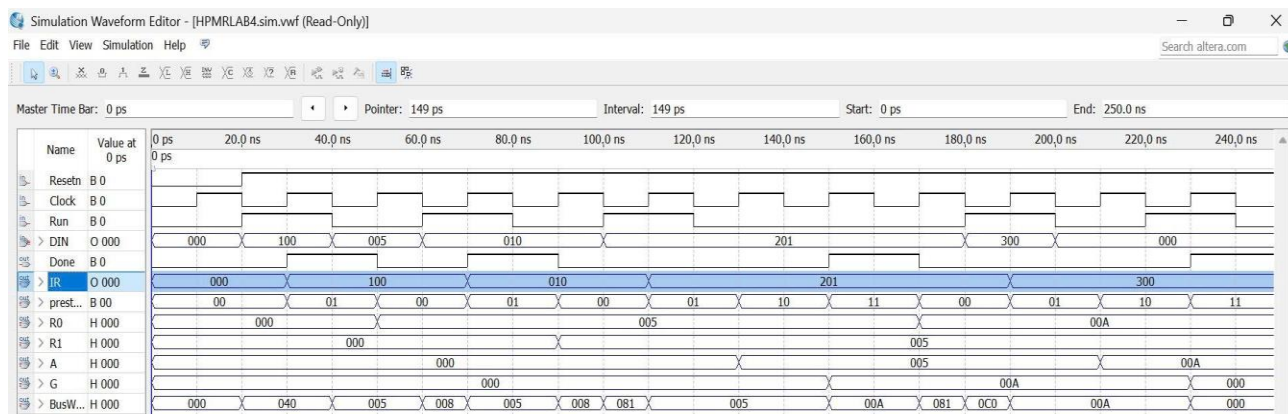
```

```

160         R_out = Yreg;
161         Gin = 1'b1;
162         AddSub_ctrl = 1'b1; // Sub
163     end
164     default: ;
165 endcase
166
167 T3:
168     case (I)
169         3'b010: begin
170             G_out = 1'b1;
171             R_in = Xreg;
172             Done = 1'b1;
173         end
174         3'b011: begin
175             G_out = 1'b1;
176             R_in = Xreg;
177             Done = 1'b1;
178         end
179         default: ;
180     endcase
181     default: ;
182 endcase
183 end
184     assign prestate = Tstep_Q;
185 endmodule

```

### 3. Kết quả mô phỏng sóng (Waveform)



Hình 2: Kết quả mô phỏng sóng của bộ xử lý đơn giản

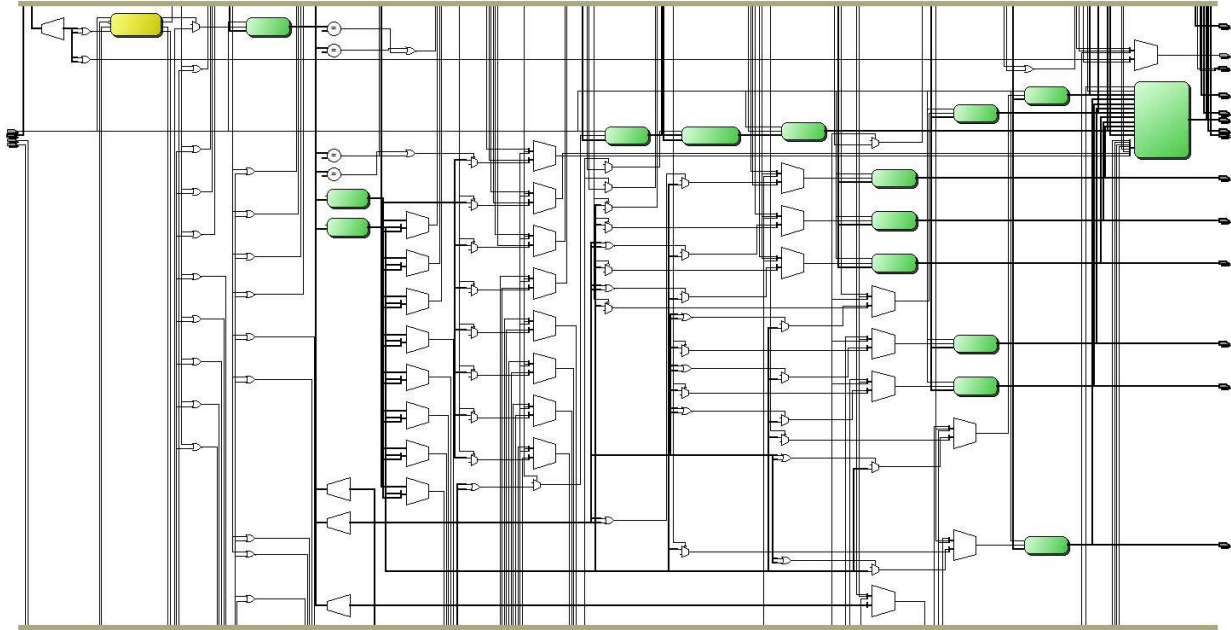
#### Nhận xét kết quả mô phỏng:

Dựa trên giản đồ xung, bộ xử lý đã thực thi chính xác trình tự các lệnh:

- Lệnh mvi (20ns - 60ns): Mã lệnh 100 và dữ liệu 005 lần lượt được nạp, kết quả thanh ghi R0 cập nhật giá trị 005 thành công.
- Lệnh mv (100ns): Mã lệnh 010 kích hoạt việc sao chép dữ liệu từ R0 sang R1, xác nhận qua việc R1 nhận giá trị 005.

- Lệnh add (140ns): Mã lệnh 201 thực hiện phép cộng ( $R0 + R1$ ), kết quả 00A (10 thập phân) xuất hiện tại thanh ghi G ở thời điểm 180ns.
- Hoạt động ổn định: Tín hiệu Done lên mức cao sau mỗi chu trình lệnh cho thấy máy trạng thái chuyển đổi đúng thiết kế trước khi bước sang lệnh sub tại 200ns.

#### 4. RTL Viewer

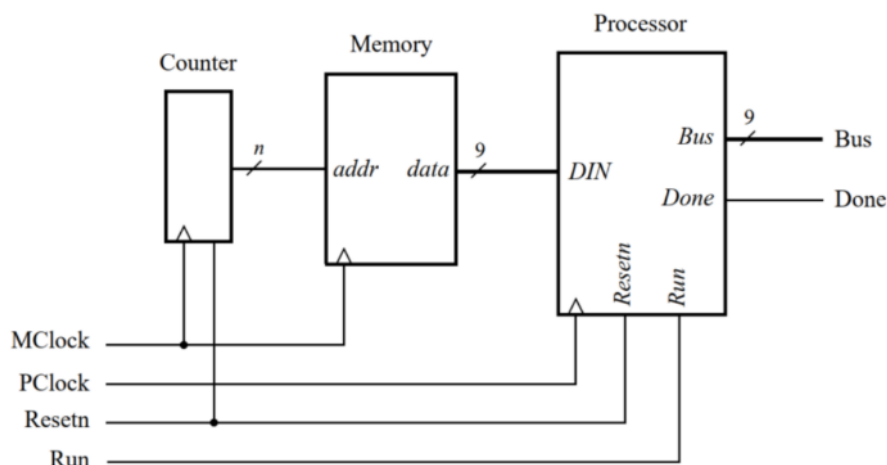


Hình 3: Sơ đồ RTL của bộ xử lý đơn giản

## EXPERIMENT 2: Design and implement a simple processor with memory

### 1. Yêu cầu bài toán

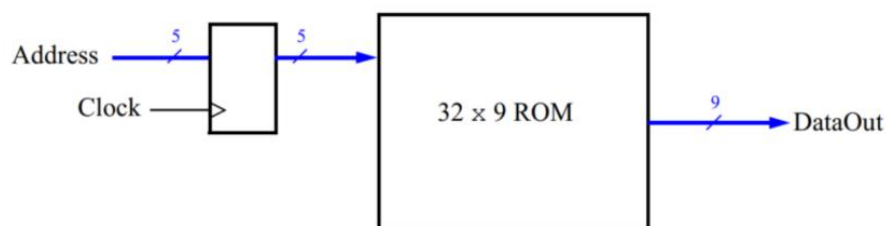
Mở rộng mạch từ Thí nghiệm 1 thành mạch trong Hình 4, trong đó một module bộ nhớ và bộ đếm được kết nối với bộ xử lý. Bộ đếm được sử dụng để đọc nội dung của các địa chỉ liên tiếp trong bộ nhớ, và dữ liệu này được cung cấp cho bộ xử lý dưới dạng một luồng lệnh. Để đơn giản hóa thiết kế và thử nghiệm mạch này, chúng tôi đã sử dụng các tín hiệu xung nhịp riêng biệt, PClock và MClock, cho bộ xử lý và bộ nhớ.



Hình 4: Sơ đồ kết nối bộ xử lý với bộ nhớ và bộ đếm

### 2. Thiết kế

#### a. Module memory32x9 (ROM)



Hình 5: Sơ đồ khối ROM 32 x 9 với thanh ghi địa chỉ

Dùng để lưu trữ chương trình (tập hợp các lệnh mã máy) để bộ xử lý thực thi. Module này được thiết kế với kích thước 32x9 (32 từ nhớ, mỗi từ 9-bit). Nó nhận tín hiệu đầu vào là địa chỉ (addr) từ bộ đếm. Dựa trên địa chỉ nhận được, nó xuất dữ liệu (data) là mã lệnh 9-bit tương ứng tại địa chỉ đó (ví dụ: mv R0, R0, add R3, R7...) ra ngõ vào DIN của bộ xử lý. Dữ liệu được khởi tạo sẵn bằng các lệnh gán (rom[0] = ..., rom[1] = ...).

```

1 module memory32x9 (
2     input logic [4:0] addr,
3     output logic [8:0] data
4 );
5
6 // ROM 32 hàng × 9 bit
7 logic [8:0] rom [0:31];
8
9 // Khởi tạo nội dung ROM tại đây
10 initial begin
11
12     rom[0] = 9'b000000000; // mv R0,R0
13     rom[1] = 9'b000001100; // mv R1,R4
14     rom[2] = 9'b000011010; // mv R3,R2
15     rom[3] = 9'b001010000; // mvi R2,#5
16     rom[4] = 9'b001100000; // mvi R4,#-6
17     rom[5] = 9'b010011111; // add R3,R7
18     rom[6] = 9'b010010000; // add R2,R0
19     rom[7] = 9'b011101110; // sub R5,R6
20     rom[8] = 9'b010101111; // add R5,R7
21     rom[9] = 9'b011011111; // sub R3,R7
22     rom[10] = 9'b011001000; // sub R1,R0
23     rom[11] = 9'b010101110; // add R5,R6
24     rom[12] = 9'b010011001; // add R3,R1
25     rom[13] = 9'b001101000; // mvi R5,#-5
26     rom[14] = 9'b000111101; // mv R7,R5
27     rom[15] = 9'b000010011; // mv R2,R3
28     rom[16] = 9'b000010111; // mv R2,R7
29     rom[17] = 9'b000101111; // mv R5,R7
30     rom[18] = 9'b010011110; // add R3,R6
31     rom[19] = 9'b011011110; // sub R3,R6
32     rom[20] = 9'b000011110; // mv R3,R6
33     rom[21] = 9'b010111101; // add R7,R5
34     rom[22] = 9'b011111101; // sub R7,R5
35     rom[23] = 9'b010010011; // add R2,R3
36     rom[24] = 9'b011010011; // sub R2,R3
37     rom[25] = 9'b000011101; // mv R3,R5
38     rom[26] = 9'b001011000; // mv R3,#5
39     rom[27] = 9'b010011101; // add R3,R5
40     rom[28] = 9'b011011101; // sub R3,R5
41     rom[29] = 9'b000100110; // mv R4,R6
42     rom[30] = 9'b010100110; // add R4,R6
43     rom[31] = 9'b011100110; // sub R4,R6
44 end
45
46 // Output read only
47 assign data = rom[addr];
48
49 endmodule

```

#### b. Module counter5

Đóng vai trò như một bộ đếm chương trình (Program Counter - PC) bên ngoài.

- Nó tạo ra chuỗi địa chỉ tuần tự từ 0 đến 31.
- Tại mỗi cạnh lên của xung nhịp MClock, giá trị ngõ ra Q tăng thêm 1 đơn vị.
- Ngõ ra Q này được kết nối trực tiếp vào ngõ vào addr của module bộ nhớ memory32x9 để điều khiển việc đọc lệnh lần lượt theo thứ tự.
- Khi bộ đếm đạt đến giá trị giới hạn (31), nó sẽ tự động quay về 0.

```

1 module counter5(
2     input logic MClock,
3     input logic resetn,
4     output logic [4:0] Q);
5 always_ff @(posedge MClock, negedge resetn)
6     begin
7         if(!resetn) Q <= 5'd0; else
8             if(Q >= 31) Q <= 5'd0; else
9                 Q <= Q + 5'd1;
10    end
11 endmodule

```

### c. Module proc (Top – level)

Module này đóng vai trò là khối tích hợp hệ thống:

- Kết nối phần cứng: Ghép nối 3 thành phần rời rạc là Bộ đếm (Counter), bộ nhớ (ROM), và bộ xử lý (CPU) thành một máy tính hoàn chỉnh.
- Tự động hóa luồng lệnh: Thiết lập cơ chế để CPU tự động nhận lệnh từ ROM thông qua địa chỉ do Bộ đếm cung cấp, thay vì phải nạp thủ công từng lệnh.
- Hỗ trợ kiểm tra (Debug): Trích xuất toàn bộ tín hiệu nội bộ (các thanh ghi R0 - R7, IR, BusWires...) ra các cổng Output để người dùng quan sát và kiểm tra lỗi trên giản đồ sóng.

```

1 module proc (
2     input logic MClock,
3     input logic PClock,
4     input logic Reset,
5     input logic Run,
6     output logic Done,
7     output logic [4:0] PC,
8     output logic [8:0] IR, A, G,
9     output logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7,
10    output logic [8:0] BusWires,
11    output logic [1:0] Tstep
12 );
13
14
15 counter5 COUNTER(
16     .MClock (MClock),
17     .resetn (Reset),

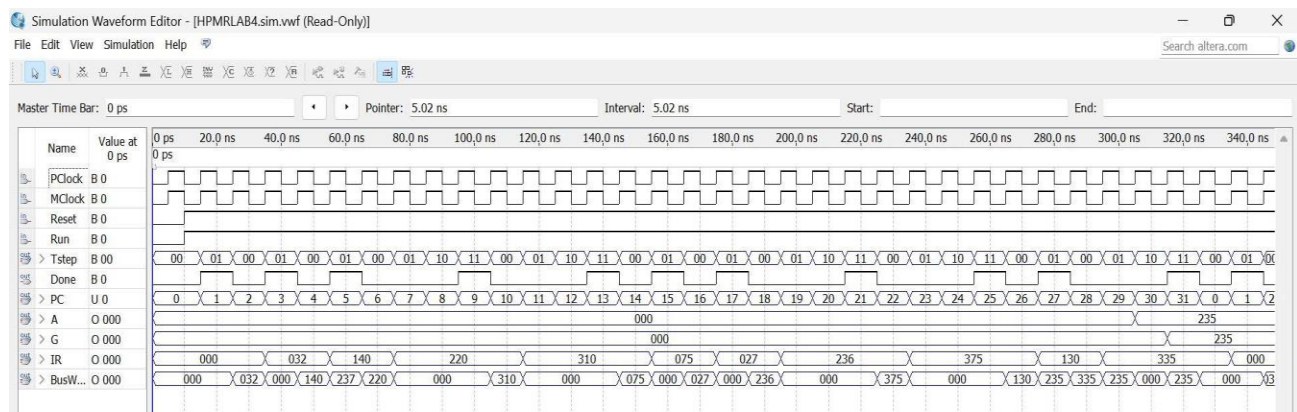
```

```

18 .Q (PC)
19 );
20
21 logic [8:0] DIN;
22
23 memory32x9 MEMORY (
24 .addr (PC),
25 .data (DIN)
26 );
27
28 CPU PROCESSOR(
29 .DIN (DIN),
30 .Resetn (Reset),
31 .Clock (PClock),
32 .Run (Run),
33 .Done (Done),
34 .BusWires (BusWires),
35 .R0(R0), .R1(R1), .R2(R2), .R3(R3),
36 .R4(R4), .R5(R5), .R6(R6), .R7(R7),
37 .IR(IR), .A(A), .G(G),
38 .prestate(Tstep)
39 );
40
41 endmodule

```

### 3. Kết quả mô phỏng sóng (Waveform)



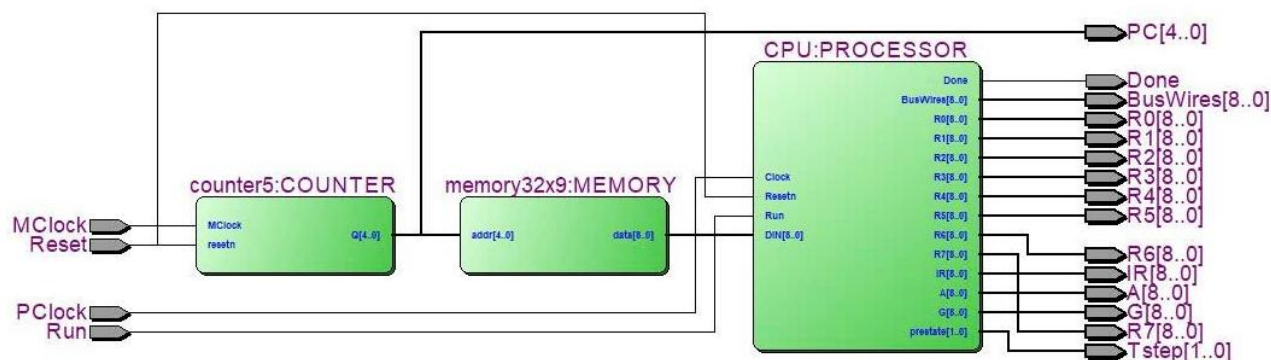
Hình 6: Kết quả mô phỏng sóng của bộ xử lý đơn giản có bộ nhớ

#### Nhận xét kết quả mô phỏng:

- Hệ thống hoạt động chính xác với hai xung nhịp riêng biệt. MClock điều khiển bộ đếm chương trình (PC) tăng tuần tự (0, 1, 2, 3...), qua đó địa chỉ được gửi tới bộ nhớ để nạp mã lệnh tương ứng vào thanh ghi IR.
- Chu trình thực thi lệnh (FSM): Biến trạng thái Tstep điều phối nhịp hoạt động cho bộ xử lý theo xung PClock:
- + Với lệnh đơn giản (ví dụ tại PC = 2, IR = 032<sub>8</sub>), Tstep chỉ cần 2 nhịp (00 - 01) để hoàn thành.

- + Với lệnh tính toán phức tạp (ví dụ tại PC = 4, IR = 237<sub>8</sub>), Tstep đếm đủ 4 nhịp (00 - 01 - 10 - 11) để thực hiện phép toán ALU.
- Tín hiệu hoàn tất: Tín hiệu Done luôn bật lên mức cao (1) tại nhịp cuối cùng của mỗi lệnh, báo hiệu bộ xử lý đã sẵn sàng cho chu kỳ nạp lệnh tiếp theo từ bộ nhớ.

#### 4. RTL Viewer



Hình 7: Sơ đồ RTL của bộ xử lý đơn giản có bộ nhớ

#### Nhận xét sơ đồ RTL:

Sơ đồ hiển thị rõ ba khối chức năng chính gồm bộ đếm địa chỉ counter5, bộ nhớ chương trình memory32x9 và bộ xử lý trung tâm CPU. Tín hiệu ngõ ra Q từ counter5 được kết nối trực tiếp đến ngõ vào addr của bộ nhớ, đảm bảo việc truy xuất lệnh tuần tự. Dữ liệu lệnh từ ngõ ra data của bộ nhớ được dẫn vào ngõ DIN của bộ xử lý để thực thi.

Bên trong khối CPU, các thanh ghi quan trọng như R0 – R7, A, G, IR cùng hệ thống BusWires đều được tổng hợp chính xác theo thiết kế và đưa ra các cổng xuất (output ports) nhằm phục vụ mục đích kiểm tra.

## KẾT LUẬN

Qua bài tập này, nhóm đã hoàn thành việc thiết kế và hiện thực một bộ xử lý đơn giản 9-bit sử dụng ngôn ngữ SystemVerilog qua hai giai đoạn:

1. Xây dựng lõi xử lý: Thiết kế thành công khối đường dữ liệu (Datapath) và khối điều khiển (FSM) để thực thi chính xác 4 lệnh cơ bản (mv, mvi, add, sub).
2. Tích hợp hệ thống: Mở rộng mạch với bộ đếm chương trình (Counter) và bộ nhớ (ROM), cho phép bộ xử lý tự động nạp và thực thi chuỗi lệnh liên tiếp.

Kết quả kiểm tra từ mô phỏng dạng sóng (Waveform) và sơ đồ tổng hợp (RTL Viewer) hoàn toàn trùng khớp với lý thuyết. Điều này khẳng định hệ thống hoạt động ổn định, kiến trúc phần cứng được tối ưu hóa và đáp ứng đúng yêu cầu của bài toán thiết kế vi mạch số cơ bản.